

Skill 11:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/wait.h>

#define HISTORY_SIZE 5
#define MAX_COMMAND 100
#define MAX_PIPELINE 3

/* =====
   HISTORY BUFFER
   ===== */

char history[HISTORY_SIZE][MAX_COMMAND];
int history_count = 0;

/* Store command in history */
void add_history(char *command)
{
    if (history_count < HISTORY_SIZE)
    {
        strcpy(history[history_count], command);
        history_count++;
    }
    else
    {
        /* Handle capacity limit */

        for (int i = 1; i < HISTORY_SIZE; i++)
        {
            strcpy(history[i - 1], history[i]);
        }

        strcpy(history[HISTORY_SIZE - 1], command);
    }
}
```

```

/* Retrieve history entry */
void retrieve_history(int index)
{
    if (index < 0 || index >= history_count)
    {
        printf("Invalid history index.\n");
        return;
    }

    printf("History entry %d: %s\n",
           index + 1,
           history[index]);
}

/* Display history */
void display_history()
{
    printf("\n===== COMMAND HISTORY =====\n");

    for (int i = 0; i < history_count; i++)
    {
        printf("%d  %s\n", i + 1, history[i]);
    }
}

/* =====
PIPELINE STRUCTURE
===== */

typedef struct
{
    |   char command[MAX_COMMAND];

```

```

/* =====
EXECUTE PIPELINE
===== */

void execute_pipeline(Pipeline *pipeline)
{
    int pipes[MAX_PIPELINE - 1][2];
    pid_t pids[MAX_PIPELINE];

    printf("\nExecuting pipeline...\n");

    /* Create pipes */

    for (int i = 0; i < pipeline->count - 1; i++)
    {
        if (pipe(pipes[i]) == -1)
        {
            perror("pipe");
            return;
        }
    }

    /* Create child processes */

    for (int i = 0; i < pipeline->count; i++)
    {
        pids[i] = fork();

        if (pids[i] < 0)
        {
            perror("fork");
            return;
        }

        if (pids[i] == 0)
        {
            /* Connect input from previous command */

```

```

/* =====
EXECUTE PIPELINE
===== */

void execute_pipeline(Pipeline *pipeline)
{
    int pipes[MAX_PIPELINE - 1][2];
    pid_t pids[MAX_PIPELINE];

    printf("\nExecuting pipeline...\n");

    /* Create pipes */

    for (int i = 0; i < pipeline->count - 1; i++)
    {
        if (pipe(pipes[i]) == -1)
        {
            perror("pipe");
            return;
        }
    }

    /* Create child processes */

    for (int i = 0; i < pipeline->count; i++)
    {
        pids[i] = fork();

        if (pids[i] < 0)
        {
            perror("fork");
            return;
        }

        if (pids[i] == 0)
        {
            /* Connect input from previous command */

```

MAIN

===== */

```
int main()
{
    printf("=====\n");
    printf("HISTORY BUFFER AND PIPELINE PRACTICAL\n");
    printf("=====\n");

    /* =====
       HISTORY TEST
       ===== */

    printf("\n1. Adding commands to history...\n");

    add_history("ls");
    add_history("pwd");
    add_history("cd /tmp");
    add_history("echo Hello");
    add_history("date");

    display_history();

    /* Retrieve entry */

    printf("\n2. Retrieving history entry:\n");

    retrieve_history(2);

    /* Capacity test */

    printf("\n3. Testing history capacity limit:\n");

    add_history("whoami");
```

```
add_pipeline_command(&pipeline, "ls");
add_pipeline_command(&pipeline, "sort");
add_pipeline_command(&pipeline, "wc");

/* Display pipeline */

printf("\n5. Pipeline structure:\n");

display_pipeline(&pipeline);

/* Validate pipeline layout */

printf("\n6. Pipeline validation:\n");

if (pipeline.count == 3)
    printf("Pipeline layout: PASS\n");
else
    printf("Pipeline layout: FAIL\n");

/* Execute pipeline */

printf("\n7. Pipeline execution:\n");

execute_pipeline(&pipeline);

printf("\n=====\n");
printf("PRACTICAL COMPLETED\n");
printf("=====\n");

return 0;
```

```
}
```

```
bhanu@DESKTOP-DE32HQ8:~$ cd ~
mkdir -p ~/history_pipeline
cd ~/history_pipeline
bhanu@DESKTOP-DE32HQ8:~/history_pipeline$ pwd
/home/bhanu/history_pipeline
bhanu@DESKTOP-DE32HQ8:~/history_pipeline$ nano history_pipeline.c
bhanu@DESKTOP-DE32HQ8:~/history_pipeline$ gcc history_pipeline.c -o history_pipeline
bhanu@DESKTOP-DE32HQ8:~/history_pipeline$ ls
history_pipeline  history_pipeline.c
bhanu@DESKTOP-DE32HQ8:~/history_pipeline$ ./history_pipeline
=====
HISTORY BUFFER AND PIPELINE PRACTICAL
=====

1. Adding commands to history...

===== COMMAND HISTORY =====
1  ls
2  pwd
3  cd /tmp
4  echo Hello
5  date

2. Retrieving history entry:
History entry 3: cd /tmp

3. Testing history capacity limit:

===== COMMAND HISTORY =====
1  cd /tmp
2  echo Hello
3  date
4  whoami
5  uname -a

4. History consistency test:
History buffer: PASS

=====
PIPELINE TEST
=====
```

```
=====
PIPELINE TEST
=====

5. Pipeline structure:

===== PIPELINE LAYOUT =====
[1] ls | [2] sort | [3] wc

6. Pipeline validation:
Pipeline layout: PASS

7. Pipeline execution:

Executing pipeline...
      2      2      36
Pipeline execution completed.

=====
PRACTICAL COMPLETED
=====
bhanu@DESKTOP-DE32HQ8:~/history_pipeline$ |
```